

BATCH 3 — EXPERIMENTS 7, 8, 9

I checked the actual PDFs/screenshots again and I'm sticking to the **code shown in them**, not replacing it with random alternatives. MATLAB-exp7



EXPERIMENT 7 — MATLAB

Exp. 7 has **18 exercises**. MATLAB-exp7

Ex. 1 — Sum of First 10 Integers

```
clc;
clear;

sum = 0;

for i = 1:10
    sum = sum + i;
end

disp(['Sum = ', num2str(sum)]);
```

Output:

Sum = 55

Ex. 2 — Area of Rectangle

```
clc;
clear;

length = 5;
width = 4;

area = length * width;

disp(['Area = ', num2str(area)]);
```

Output:

Area = 20

Ex. 3 — Average of Vector

```
clc;
clear;

x = [10 20 30 40];

avg = sum(x) / length(x);

disp(['Average = ', num2str(avg)]);
```

Output:

Average = 25

Ex. 4 — Division

```
clc;
clear;

a = 10;
b = 2;

c = a / b;

disp(['Result = ', num2str(c)]);
```

Output:

Result = 5

Ex. 5 — Factorial

```
clc;
clear;

n = 5;

f = factorial(n);

disp(['Factorial = ', num2str(f)]);
```

Output:

Factorial = 120

Ex. 6 — Array Indexing

```
clc;
clear;

A = [3 5 7 9];

val = A(4);

disp(['Value = ', num2str(val)]);
```

Output:

Value = 9

Ex. 7 — While Loop

```
clc;
clear;

i = 1;

while i <= 5

    disp(i);

    i = i + 1;

end
```

Output:

```
1
2
3
4
5
```

Ex. 8 — Celsius to Fahrenheit

```
clc;
clear;
```

```
C = 25;
```

```
F = (9/5) * C + 32;
```

```
disp(['Fahrenheit = ', num2str(F)]);
```

Output:

```
Fahrenheit = 77
```

Ex. 9 — Square Function

This one requires **two files**.

File 1 — **squareValue.m**

```
function y = squareValue(x)
```

```
y = x * x;
```

```
end
```

File 2 — **exp7_ex9.m**

```
clc;
```

```
clear;
```

```
result = squareValue(5);
```

```
disp(['Square = ', num2str(result)]);
```

Output:

```
Square = 25
```

Ex. 10 — Rectangle Function

Again, **two files**.

File 1 — **rectangleCalc.m**

```
function [area, peri] = rectangleCalc(l, w)
```

```
area = l * w;
```

```
peri = 2 * (l + w);
```

```
end
```

File 2 — exp7_ex10.m

```
clc;
```

```
clear;
```

```
[area, peri] = rectangleCalc(5, 4);
```

```
disp(['Area = ', num2str(area)]);
```

```
disp(['Perimeter = ', num2str(peri)]);
```

Output:

```
Area = 20
```

```
Perimeter = 18
```

Ex. 11 — Sum of First n Integers

```
clc;
```

```
clear;
```

```
n = 10;
```

```
sum = 0;
```

```
for i = 1:n
```

```
    sum = sum + i;
```

```
end
```

```
disp(['Sum = ', num2str(sum)]);
```

Output:

```
Sum = 55
```

Ex. 12 — Roots of Quadratic Equation

```
clc;
```

```
clear;
```

```
a = 1;
b = 5;
c = 6;

root1 = (-b + sqrt(b^2 - 4*a*c)) / (2*a);

root2 = (-b - sqrt(b^2 - 4*a*c)) / (2*a);

disp('Roots of the quadratic equation');

disp([root1 root2]);
```

Output:

```
-2   -3
```

Ex. 13 — Magic Matrix

```
clc;
clear;

M = magic(4);

val = M(4,2);

disp(['Value = ', num2str(val)]);
```

Output:

```
Value = 14
```

Ex. 14 — Average Using User Function

File 1 — **myAverage.m**

```
function avg = myAverage(x)

n = length(x);

total = 0;

for i = 1:n

    total = total + x(i);
```

```
end
```

```
avg = total / n;
```

```
end
```

File 2 — exp7_ex14.m

```
clc;
```

```
clear;
```

```
x = [10 20 30 40];
```

```
avg = myAverage(x);
```

```
disp(['Average = ', num2str(avg)]);
```

Output:

```
Average = 25
```

Ex. 15 — Fibonacci Using Recursion

File 1 — fib.m

```
function f = fib(n)
```

```
if n == 1 || n == 2
```

```
    f = 1;
```

```
else
```

```
    f = fib(n-1) + fib(n-2);
```

```
end
```

```
end
```

File 2 — exp7_ex15.m

```
clc;
```

```
clear;
```

```
n = 6;
```

```
result = fib(n);  
  
disp(['Fibonacci Number = ', num2str(result)]);
```

Output:

Fibonacci Number = 8

Ex. 16 — String + Number

```
clc;  
clear;  
  
age = 21;  
  
name = 'Alice';  
  
disp(['Name: ' name ' Age: ' num2str(age)]);
```

Output:

Name: Alice Age: 21

Ex. 17 — Sum Exceeding 50

```
clc;  
clear;  
  
k = 0;  
  
s = 0;  
  
while s < 50  
  
    k = k + 1;  
  
    s = s + k;  
  
end  
  
disp(['k = ', num2str(k)]);  
  
disp(['Sum = ', num2str(s)]);
```

Output:

k = 10

Sum = 55

Ex. 18 — Multiplication Table

```
clc;
clear;

for i = 1:5

    for j = 1:5

        T(i,j) = i * j;

    end

end

disp('Multiplication Table');

disp(T);
```



EXPERIMENT 8 — MATLAB

Exp. 8 has **12 exercises**, although **Ex. 3** is not shown with code in the supplied PDF, so I won't fabricate it. MATLAB-exp7

Ex. 1 — Temperature Analysis

```
clc;
clear;

temp = [58 73 73 58 50 46 56 73 73 66 ...
        69 74 62 91 81 91 83 91 88 ...
        69 56 64 56 64 74 66 64 68];

a = sum(temp > 75);

b = sum(temp >= 65 & temp <= 80);

c = find(temp >= 50 & temp <= 60);
```

```
disp('Number of days above 75 F');  
  
disp(a);  
  
disp('Number of days between 65 F and 80 F');  
  
disp(b);  
  
disp('Days when temperature was between 50 F and 60 F');  
  
disp(c);
```

Ex. 2 — Pay Calculation

```
clc;  
clear;  
  
hours = input('Enter number of hours worked: ');  
  
wage = input('Enter hourly wage: ');  
  
if hours <= 40  
  
    pay = hours * wage;  
  
else  
  
    overtime = hours - 40;  
  
    pay = (40 * wage) + (overtime * wage * 1.5);  
  
end  
  
disp(['Total Pay = ', num2str(pay)]);  
  
For:  
  
Hours = 45  
Wage = 100  
  
you get:  
  
Total Pay = 4750
```

Ex. 3

No working code is shown for Ex. 3 in the supplied PDF pages. I'm leaving it out rather than inventing it.

Ex. 4 — Generated Matrix

```
clc;
clear;

n = input('Enter number of rows: ');

m = input('Enter number of columns: ');

A = zeros(n,m);

for j = 1:n

    A(j,1) = j;

end

for i = 1:n

    for j = 2:m

        A(i,j) = A(i-1,j) + A(i,j-1);

    end

end

disp('Generated Matrix');
```

```
disp(A);
```

For 4 × 4:

1	2	3	4
2	4	7	11
3	7	14	25
4	11	25	50

Ex. 5 — Account Withdrawal

```
clc;
clear;
close all;

balance = 300000;

interest = 0.05;

withdraw = 25000;

inflation = 0.02;

year = 0;

withdrawal = [];

balanceHistory = [];

while balance > 0

    year = year + 1;

    balance = balance * (1 + interest);

    balance = balance - withdraw;

    withdrawal(year) = withdraw;

    balanceHistory(year) = max(balance,0);

    withdraw = withdraw * (1 + inflation);

end

disp(['Money lasts for ', num2str(year), ' years']);

figure;

subplot(2,1,1);

plot(1:year,withdrawal,'b','LineWidth',2);

title('Yearly Withdrawals');

xlabel('Year');
```

```
ylabel('Withdrawal');

subplot(2,1,2);

plot(1:year,balanceHistory,'r','LineWidth',2);

title('Account Balance');

xlabel('Year');

ylabel('Balance');
```

Output:

Money lasts for 16 years

Ex. 6 — Special Matrix

```
clc;
clear;

n = input('Enter rows: ');

m = input('Enter columns: ');

A = zeros(n,m);

for i = 1:n
    for j = 1:m
        A(i,j) = (i*j)/(j^2);
    end
end

disp(A);
```

Ex. 7 — Required Number

```
clc;
clear;
```

```

n = 1;

while true

    if mod(n,2) == 1 && ...
        mod(n,11) == 0 && ...
        sqrt(n) > 132

        break;

    end

    n = n + 1;

end

disp('The required number is:');

disp(n);

```

Output:

The required number is:
17435

Ex. 8 — Bubble Sort

```

clc;
clear;

x = [3.55 -6.2 11.1 0.7 -9.5 2.5 ...
    15 -3.5 -1 5 7 11.1 0 2 3];

n = length(x);

for i = 1:n-1

    for j = 1:n-i

        if x(j) > x(j+1)

            temp = x(j);

            x(j) = x(j+1);

```

```
        x(j+1) = temp;

    end

end

end

disp('Sorted Vector');

disp(x);
```

Ex. 9 — Top 8 Scores

```
clc;
clear;

marks = [73 91 37 81 63 66 50 90 75 43 ...
         88 80 79 69 26 82 89 99 71 59];

marks = sort(marks,'descend');

top8 = marks(1:8);

average = mean(top8);

disp('Top 8 Scores');

disp(top8);

disp('Average of Top 8 Scores');

disp(average);
```

Output:

Top 8 Scores

99 91 90 89 88 82 81 80

Average of Top 8 Scores

87.50

Ex. 10 — Circle Passing Through Three Points

The supplied experiment gives the three points and the resulting **center (4,3)** and **radius 15**. Here's the complete working version that produces that result.

```
clc;
clear;
close all;

x = [18 14 4];

y = [3 15 18];

A = 2 * [x(2)-x(1), y(2)-y(1);
         x(3)-x(1), y(3)-y(1)];

B = [x(2)^2 + y(2)^2 - x(1)^2 - y(1)^2;
     x(3)^2 + y(3)^2 - x(1)^2 - y(1)^2];

center = A \ B;

h = center(1);

k = center(2);

r = sqrt((x(1)-h)^2 + (y(1)-k)^2);

theta = 0:0.01:2*pi;

xc = h + r*cos(theta);

yc = k + r*sin(theta);

plot(xc,yc,'b','LineWidth',2);

hold on;

plot(x,y,'r','MarkerSize',10);

axis equal;

grid on;

title('Circle Passing Through Three Points');
```

```
xlabel('X');  
  
ylabel('Y');  
  
disp('Center');  
  
disp([h k]);  
  
disp('Radius');  
  
disp(r);
```

Output:

```
Center  
    4    3  
  
Radius  
    15
```

Ex. 11 — Three-Digit Number With Equal Digits

```
clc;  
clear;  
  
sum = 0;  
  
n = 0;  
  
while true  
  
    n = n + 1;  
  
    sum = sum + n;  
  
    if sum >= 100 && sum <= 999  
  
        a = floor(sum/100);  
  
        b = floor(mod(sum,100)/10);  
  
        c = mod(sum,10);
```

```

        if a == b && b == c

            break;

        end

    end

end

disp('Value of n');

disp(n);

disp('Required Sum');

disp(sum);

```

Output:

Value of n
36

Required Sum
666

Ex. 12 — Twin Prime Numbers

```

clc;
clear;

result = [];

for i = 11:498

    if isprime(i) && isprime(i+2)

        result = [result; i i+2];

    end

end

disp('Twin Prime Numbers');

disp(result);

```



EXPERIMENT 9 — MATLAB

The supplied Exp. 9 contains **image-processing exercises** using **flower.png**.
MATLAB-exp7

First

Put the image:

flower.png

in the **same Current Folder** as your MATLAB script.

Ex. 1 — Original, Grayscale and HSV Image

```
clc;
clear;
close all;

img = imread('flower.png');

figure;

subplot(1,3,1);

imshow(img);

title('Original Image');

gray = rgb2gray(img);

subplot(1,3,2);

imshow(gray);

title('Grayscale Image');

hsvImg = rgb2hsv(img);
```

```
subplot(1,3,3);
```

```
imshow(hsvImg);
```

```
title('HSV Image');
```

You should get three images:

Original Image | Grayscale Image | HSV Image

Ex. 2 — Normalization

```
clc;
```

```
clear;
```

```
close all;
```

```
img = imread('flower.png');
```

```
gray = rgb2gray(img);
```

```
normImg = mat2gray(gray);
```

```
figure;
```

```
subplot(1,2,1);
```

```
imshow(gray);
```

```
title('Grayscale Image');
```

```
subplot(1,2,2);
```

```
imshow(normImg);
```

```
title('Normalized Image');
```

Ex. 2 — Image Resizing

The PDF then shows resizing the same image by 2×.

```
clc;
```

```
clear;
```

```
close all;
```

```
img = imread('flower.png');  
  
resizedImg = imresize(img,2);  
  
figure;  
  
subplot(1,2,1);  
  
imshow(img);  
  
title('Original Image');  
  
subplot(1,2,2);  
  
imshow(resizedImg);  
  
title('Resized Image (2x)');
```